

Campus Connect Final Deliverable

Perez Abhuilmen Alex Cooper Tyler Geiger Kadin Matotek
CSC 468 - Cloud Computing Dr. Linh Ngo

May 5, 2025

Contents

Introduction	3
Our Vision	4
System Architecture	5
Frontend (FE)	5
Backend (BE)	6
Database (DB)	6
Cache / Session Store	6
Ingress Controller	6
Kubernetes Orchestration (Minikube on CloudLab)	7
CI/CD Pipeline and Automation	7
Detailed Backend Implementation	9
Core Infrastructure Modules	9
Feature Modules	10
Cross-Cutting Concerns	12
Security Considerations	13
Data Persistence and Modeling	16
MongoDB	16
Redis	18
Data Access	18
Conclusion: Reflections on the Learning Process	19
Appendix: Resumes	21

Contents

Introduction

Communication practices vary across West Chester University. While standard tools like email are common, they often prove inefficient for the collaborative and fast-paced nature of student projects and discussions. This inefficiency is evident when compared to specific communities within WCU, such as the Computer Science department, where platforms like Discord are heavily utilized for course communication, club activities, and general interaction.

This widespread use of Discord within CS highlights a student need for real-time communication tools. However, this adoption isn't mirrored across the university, leading to fragmented communication experiences depending on one's department or social circle. This project, Campus Connect, attempts to explore a solution by developing a centralized, real-time messaging platform concept designed for the broader WCU environment, aiming to bridge this gap.

Our Vision

The core idea behind Campus Connect is to provide a dedicated WCU messaging platform, integrating features useful for students, faculty, and staff. The goal is to offer something more tailored to university life than generic tools, potentially including features like course channels or integration with WCU systems in the future.

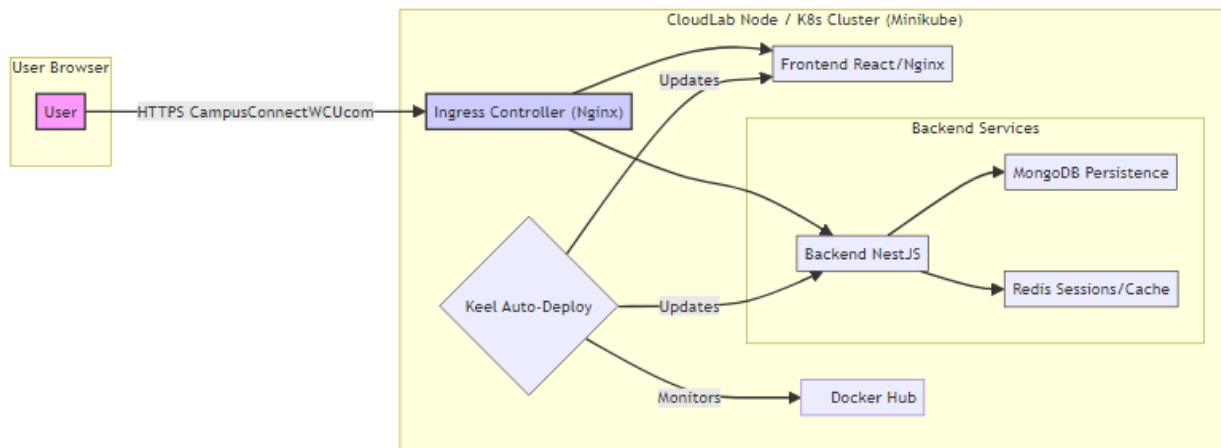
For the scope of CSC 468, our work focused on building and deploying the essential infrastructure: a backend API with real-time WebSocket capabilities, a functional frontend interface, persistent database storage, and an automated CI/CD pipeline for deployment onto CloudLab infrastructure. The current implementation demonstrates these technical components working together. It establishes basic real-time chat functionality but does not yet include many of the advanced features (like dedicated classroom spaces or sophisticated group management) found in established platforms like Slack or Discord.

The primary outcome of this phase is a demonstration of feasibility – showing that such a system *can* be built and deployed using the chosen technologies (Kubernetes, Helm, NestJS, React). We registered the domain `CampusConnectWCU.com` via IONOS, and the deployment scripts automate the process of pointing this domain to the active CloudLab instance's IP address. While significant further development is needed to make it a competitive alternative, this project serves as a foundational step and learning experience in building scalable, cloud-native applications for a university setting. We intend to continue developing it independently after this semester.

System Architecture

Campus Connect is designed as a set of distinct services (microservices) running in Docker containers. These containers are managed within a Kubernetes cluster created using Minikube on a CloudLab virtual machine. Helm is used to define and manage the Kubernetes configurations, Skaffold helps with the build process, and GitHub Actions automate testing and deployment triggers. This approach promotes modularity, scalability, and maintainability, allowing different components of the system to be developed, deployed, and scaled independently.

The main runtime components and their interactions are illustrated below:



The key runtime components and their interactions are detailed below:

Frontend (FE)

- **Technology:** Developed using the React JavaScript library.
- **Functionality:** Provides the user interface (UI) accessible via a web browser. Users interact with the frontend to register, log in, view message history, and send/receive real-time messages.
- **Deployment:** The React application is built into static HTML, CSS, and JavaScript assets. These assets are served by an Nginx web server running in its own Docker container within the Kubernetes cluster.

Backend (BE)

- **Technology:** Built with the NestJS framework, leveraging Node.js and TypeScript.
- **Functionality:** Acts as the central brain of the application.
 - It exposes a RESTful API for handling user authentication (signup, login), fetching historical data (messages, channels), and other non-real-time operations.
 - It manages real-time communication using WebSockets via NestJS Gateways. This allows for bidirectional communication between the server and connected clients for instant message delivery.
 - It handles business logic and interacts with the database (MongoDB) and cache (Redis) to persist and retrieve data.
- **Deployment:** Runs as a distinct Docker container within the Kubernetes cluster, managed by a Kubernetes Deployment resource.

Database (DB)

- **Technology:** MongoDB, a NoSQL document database.
- **Functionality:** Provides persistent storage for application data, including user credentials (securely hashed), user profiles, chat messages, and channel information.
- **Deployment:** Runs as a separate container within the Kubernetes cluster, managed by a StatefulSet and utilizes PersistentVolumes/PersistentVolumeClaims for data persistence across pod restarts.

Cache / Session Store

- **Technology:** Redis, an in-memory data structure store.
- **Functionality:** Used for:
 - **Session Management:** Storing user session identifiers to keep users logged in across multiple requests.
 - **Caching:** Caching frequently accessed data to reduce latency and database load.
- **Deployment:** Runs as another containerized service within Kubernetes, managed by a Deployment or StatefulSet.

Ingress Controller

- **Technology:** Nginx Ingress Controller.
- **Functionality:** Acts as the single entry point for all external HTTP/HTTPS traffic for CampusConnectWCU.com into the Kubernetes cluster. It routes incoming requests to the appropriate internal service (Frontend or Backend) based on configured path rules.
- **Features:** Handles TLS termination (decrypting HTTPS traffic) using configured certificates/secrets, load balancing across service pods, and defining path-based routing.
- **Deployment:** Runs as dedicated pods within the Kubernetes cluster.

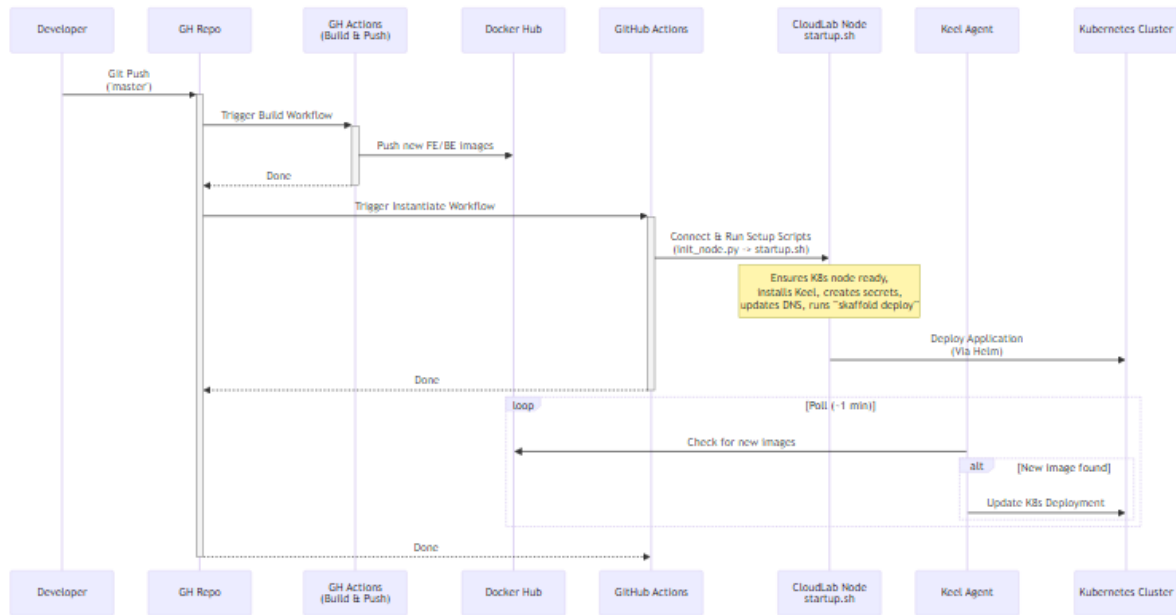
Kubernetes Orchestration (Minikube on CloudLab)

- **Platform:** Minikube provides a single-node Kubernetes cluster running on a CloudLab-provided virtual machine.
- **Resource Management:** All application components (Frontend, Backend, DB, Cache) and supporting infrastructure (Ingress) are defined as Kubernetes resources (Deployments, StatefulSets, Services, Ingresses, PersistentVolumeClaims, Secrets, ConfigMaps).
- **Configuration:** Helm charts template these Kubernetes resource manifests, allowing for parameterized configurations and simplifying application lifecycle management within Kubernetes via `helm` commands.

CI/CD Pipeline and Automation

- **Source Control:** The codebase resides in a GitHub repository.
- **GitHub Actions (Build & Push):** Workflows are configured to trigger automatically on pushes to the main branch. These workflows build the Docker images for the Frontend and Backend and push these images to Docker Hub.
- **GitHub Actions (Instantiate/Deploy):** Another workflow handles the deployment to the CloudLab environment. It involves:
 - Connecting to the CloudLab VM via SSH.
 - Running setup scripts (`init_node.py`, `startup.sh`) to ensure the CloudLab node is prepared (Minikube running, required tools installed).
 - Executing `skaffold deploy` (or `helm upgrade --install`) to apply the Helm charts to the Kubernetes cluster, instructing Kubernetes to use the configured images.
 - Calling the IONOS API to update the DNS A record for `CampusConnectWCU.com` to point to the CloudLab node's current public IP address.
- **Skaffold:** Used in the deployment process to coordinate applying Helm charts or Kubernetes manifests.
- **Keel:** Runs as an agent within the Kubernetes cluster, monitoring Docker Hub for new image tags matching configured policies. Upon detecting a newer image tag pushed by the build workflow, Keel automatically updates the corresponding Kubernetes Deployment, triggering a rolling update of the application pods.

The CI/CD workflow is visualized below:



Detailed Backend Implementation

The backend for Campus Connect is built using the **NestJS** framework, leveraging **TypeScript** for strong typing and a modular architecture for maintainability and scalability. The core structure, as defined in `app.module.ts`, imports various feature and infrastructure modules responsible for specific domains of functionality.

Core Infrastructure Modules

These modules provide foundational services used across the application:

1. **ConfigModule:**
 - Wraps `@nestjs/config` to load environment variables from a `.env` file (or Kubernetes secrets/configmaps in production).
 - Uses the **Joi** library via `envValidationSchema` for startup validation of required variables (e.g., database connection details, ports, secrets).
 - Provides the **ConfigService** for strongly-typed access to configuration values.
2. **DatabaseModule:**
 - Manages connections to persistent stores.
 - Configures **Mongoose** for MongoDB interaction (`mongoProvider`).
 - Sets up a **Redis** client (`redisProvider`) using connection details from `ConfigService`.
 - Exports Mongoose models and the Redis client (`REDIS_CLIENT`) for injection.
 - Provides the **DatabaseService** with utility functions for health checks (`pingRedis`, `mongoHealthCheck`).
3. **EncryptionModule:**
 - Provides the **EncryptionService** for symmetric encryption/decryption (AES-256-CBC) using Node.js's built-in `crypto` library.
 - Derives the encryption key from the `ENCRYPTION_KEY` environment variable via `ConfigService`.
 - Primarily used by `SessionModule` to secure session data in Redis.
4. **SessionModule:**
 - Handles user session management.
 - Configures and applies the **SessionMiddleware** globally.
 - Uses `express-session` with `connect-redis` as the store.
 - Custom serializer leverages **EncryptionService** to encrypt/decrypt session data.

stored in Redis.

- Provides the **SessionService** with helpers for session interaction (e.g., `saveSession`, `destroySession`, `getSessionFromRawCookie`, `getUserIdFromRawCookie`).
- Cookie behavior (secure, httpOnly, sameSite, maxAge) is configured via `ConfigService`.

5. **CampusModule:**

- Manages campus information.
- Defines the **Campus schema** (name, city, state, domain).
- Uses **CampusService** / **CampusRepository** for database interactions.
- Includes logic (`onModuleInit` in `AppModule` calling `campusService.seedCampuses`) to seed predefined campuses (from `campus.constants.ts`) on startup.
- Used by `UsersModule` to link users to campuses.

6. **HealthModule:**

- Exposes a `/health` REST endpoint via **HealthController**.
- The **HealthService** uses **DatabaseService** to perform connectivity checks against MongoDB and Redis.

Feature Modules

These modules implement the core business logic and user-facing features:

1. **UsersModule:** Central to user management.

- **Schema (User):** Defines MongoDB document structure (email, username, hashed-Password, major, fullName, campus reference, friend/request/blocked IDs).
- **Repository (UsersRepository):** Handles direct `userModel` interaction (e.g., `createUser`, `findByEmail`, `findByIdentifier`, `updateUser`, `findAllExcept`). Uses `.populate('campus', 'name')`.
- **Service (UsersService):** Contains business logic.
 - `registerUser`: Checks duplicates, validates campus, hashes password (`bcrypt.hash`), calls repository.
 - `validateUserCredentials`: Finds user, verifies password (`bcrypt.compare`).
 - `updateUser`: Handles profile updates, validates uniqueness, checks campus, handles password changes securely.

2. **AuthModule:** Manages authentication and authorization.

- **Controller (AuthController):** Defines REST API endpoints under `/api/auth`:
 - `POST /signup`, `POST /login`: User registration and login (calls `AuthService`).
 - `POST /logout`: Destroys session (calls `AuthService/SessionService`). Requires `AuthGuard`.
 - `GET /session`: Checks for a valid session.
 - `POST /exists`: Checks if a user exists.
 - `GET /profile`, `POST /update`: Retrieve/update logged-in user's profile. Requires `AuthGuard`. Strips sensitive data from response.
- **Service (AuthService):** Orchestrates authentication, interacting with `UsersService` and `SessionService`.

- **Guard (AuthGuard):** Implements `CanActivate` to protect routes by checking `request.session.userId`.
 - **Decorator (@CurrentUser):** Custom decorator to extract `request.session.userId` in controllers.
3. **ChannelsModule:** Handles real-time chat (DMs and Group chats).
- **Controller (ChannelsController):** Defines REST endpoints under `/api/channels` (fetching channels/metadata, messages, sending/editing/deleting messages, getting/creating DMs, searching, creating groups, marking read). Most protected by `ChannelsGuard`.
 - **Guard (ChannelsGuard):** Protects routes by verifying the authenticated user is a participant in the target channel (via `ChannelsService`).
 - **Gateway (ChannelsGateway):** Manages WebSocket connections (`/channels` namespace).
 - Authenticates connections using `connect.sid` cookie via `SessionService`.
 - Clients join channel rooms (`client.join(channelId)`).
 - Listens for `message.sent` and `channel.read` events from `EventEmitter2`.
 - Broadcasts `messageReceived` and `channelRead` events to relevant `channelId` rooms (`server.to(channelId).emit(...)`).
 - **Services (ChannelsService, MessageService):**
 - **MessageService:** Handles message CRUD via `MessageRepository`. Emits `message.sent` event via `EventEmitter2` after saving a new message.
 - **ChannelsService:** Manages channel logic (fetching, getting/creating DMs, metadata, searching, marking read) via `ChannelsRepository`. Emits `channel.read` event via `EventEmitter2` when marking as read.
 - **Schemas (Channel, Message):** Define MongoDB structure for channels (type, name, participants) and messages (`channelId`, `senderId`, `senderName`, `content`, `edited`, `readBy`).
4. **PostsModule:** Handles campus-wide bulletin board posts.
- **Controller (PostController):** Defines REST endpoints under `/api/posts` (creating, retrieving by campus, liking/unliking). Uses `AuthGuard`. Implicitly checks user's campus membership.
 - **Gateway (PostsGateway):** Manages WebSocket connections (`/posts` namespace).
 - Clients join campus-specific rooms (`client.join(campusId)`).
 - Listens for `post.created` and `post.liked` events via `EventEmitter2`.
 - Broadcasts `postCreated/postLiked` events to the specific `campusId` room.
 - **Service (PostService):** Contains business logic.
 - `createPost`: Validates user's campus, saves via `PostRepository`, emits `post.created`.
 - `likePost`: Toggles like status via `PostRepository`, emits `post.liked`.
 - **Repository (PostRepository):** Interacts with `postModel`. Handles creation, fetching by campus (sorted), and toggling likes.
 - **Schema (Post):** Defines post structure (`campusId`, `senderId`, `senderName`, `senderMajor`, `content`, `hashtags`, `likes`).
5. **FriendsModule:** Manages user social connections.

- **Controller (`FriendsController`):** Defines REST endpoints under `/api/friends` (requesting, accepting, denying, removing, canceling, blocking, unblocking, fetching lists, getting suggestions). Uses `AuthGuard`.
 - **Service (`FriendsService`):** Implements friendship logic.
 - Interacts heavily with `UsersRepository` to update relationship arrays (`friends`, `friendRequests`, etc.) on user documents.
 - Calls `NotificationsService` methods after actions (e.g., sending/accepting requests) to trigger real-time updates.
 - `suggestFriends`: Provides potential friends based on mutuals, campus, and major.
6. **NotificationsModule:** Facilitates real-time, non-chat notifications.
- **Gateway (`NotificationsGateway`):** Manages WebSocket connections (`/notifications` namespace).
 - Authenticates connections via cookie.
 - **Maintains a map (`clients`) linking `userId` to active `Socket` instances.**
 - Listens for `notification.sent` events via `EventEmitter2`.
 - Looks up the target `userId` in the `clients` map and emits a `notification` event directly to that user's socket if found.
 - **Service (`NotificationsService`):** Acts as an intermediary.
 - Called by other services (e.g., `FriendsService`) via methods like `notifyFriendRequest`.
 - Constructs notification payload and emits the generic `notification.sent` event via `EventEmitter2`, including the `targetUserId`.

Cross-Cutting Concerns

Beyond specific modules, several patterns and tools are applied globally:

- **Middleware:** Configured in `main.ts` and module files.
 - `LoggerMiddleware`: Logs basic request/response info.
 - `SessionMiddleware`: Handles session loading/saving.
- **Filters:**
 - `HttpExceptionFilter`: Applied globally in `main.ts` to catch `HttpExceptions` and format consistent JSON error responses.
- **Event Emitter (`EventEmitter2`):**
 - Used as an internal event bus to decouple services from WebSocket gateways.
 - Services emit events (e.g., `message.sent`, `post.created`, `notification.sent`).
 - Gateways subscribe to these events to push real-time updates, promoting separation of concerns.

This modular structure, combined with clear separation of concerns (Controller, Service, Repository, Gateway) and the use of dependency injection and event emission, forms the foundation of the Campus Connect backend.

Security Considerations

Security was a key consideration throughout the design and implementation of Campus Connect, addressed at multiple layers from the application code to the deployment infrastructure.

1. Authentication and Authorization:

- **User Authentication:** User login relies on validating credentials against hashed passwords stored in MongoDB. The `bcryptjs` library is used (`UserService`) for hashing passwords with a salt (work factor 10).
- **Session Management:** Secure session management is implemented using `express-session` backed by `connect-redis`. Session IDs are stored in `httpOnly`, `secure` (conditionally based on `COOKIE_SECURE` env var), and `sameSite` cookies to mitigate CSRF and XSS risks. Session data itself is encrypted before being stored in Redis using the `EncryptionService` (AES-256-CBC).
- **API Protection:** Most API endpoints are protected using the custom `AuthGuard`, which verifies the presence of a valid `userId` in the decrypted session data associated with the request's cookie.
- **WebSocket Authentication:** WebSocket connections to the `/channels`, `/notifications`, and `/posts` namespaces are authenticated by parsing the `connect.sid` cookie from the handshake request (`cookie.parse`, `cookie-signature.unsign`) and validating the corresponding session using the `SessionService`. Unauthenticated connections are disconnected.
- **Resource Authorization:** Beyond authentication, authorization checks are implemented:
 - `ChannelsGuard`: Ensures users can only access channels they are participants in.
 - `Service Logic`: Specific checks within services prevent unauthorized actions, such as ensuring only the sender can edit/delete their messages (`MessageService`) or users can only get/create posts within their own campus (`PostService`).

2. Data Security:

- **Password Hashing:** As mentioned, `bcryptjs` is used for securely storing user passwords.
- **Session Encryption:** Session data stored in Redis is encrypted using AES-256-CBC (`EncryptionService`) before being written to Redis, as configured in the `SessionMiddleware`'s custom serializer. The Redis instance itself is password-protected, with the password managed via a Kubernetes secret (`REDIS_PASSWORD`).

in `campus-connect-config-secrets`) and mounted into the Redis deployment (`helm/templates/redis-deployment.yaml`).

- **TLS/HTTPS:** External communication is secured using HTTPS. The `startup.sh` script handles creating a Kubernetes TLS secret (`campusconnect-tls`) from base64 encoded certificate and key provided via GitHub Secrets (`TLS_CERT_B64`, `TLS_KEY_B64`). The Nginx Ingress resource (`helm/templates/ingress.yaml`) is configured to use this secret for TLS termination for the `campusconnectwcu.com` domain.
- **Data Exposure:** Sensitive data like password hashes are explicitly excluded from API responses (e.g., in `AuthController` profile endpoints).

3. Infrastructure and Deployment Security:

- **Secrets Management:** Sensitive configuration values (API keys, database passwords, session/encryption secrets) are managed using GitHub Secrets. These are passed securely to the CloudLab node during the deployment workflow (`instantiate.yaml`) as environment variables and then used by `startup.sh` to create Kubernetes `Secret` objects (`campus-connect-config-secrets`, `campusconnect-tls`). Application containers mount these secrets via `envFrom` directives in their deployments (`helm/templates/backend-deployment.yaml`, `helm/templates/redis-deployment.yaml`), avoiding hardcoding secrets in code or configuration files. Temporary TLS files created during secret generation are securely deleted (`sudo rm -f`).
- **Container Security:** Production Docker images (`Dockerfile.backend.prod`, `Dockerfile.frontend.prod`) are built using minimal Alpine base images (`node:20-alpine`, `nginx:alpine`) to reduce the attack surface. Multi-stage builds are used for the frontend to exclude build dependencies. Crucially, both backend (`appuser`) and frontend (`nginx`) containers run as non-root users, limiting privileges in case of compromise. Only production dependencies are installed (`npm install --production`).
- **Network Security (Kubernetes):** Kubernetes Network Policies are defined in the Helm chart (`helm/templates/`) to enforce network segmentation. A `default-deny-all` policy restricts all ingress/egress by default. Specific policies then explicitly allow necessary traffic: Ingress controller to Frontend/Backend, Backend egress to Mongo/Redis, and Mongo/Redis ingress only from the Backend. This limits the potential lateral movement of an attacker within the cluster.
- **Network Security (Node):** The `startup.sh` script uses `socat` and `minikube tunnel` with `sudo` to forward host ports 80 and 443 to the Minikube Ingress service, ensuring external traffic reaches the application.
- **CI/CD Security:** Secrets are managed via GitHub Secrets. The deployment process uses a dedicated, non-root user (`ccuser`) on the CloudLab node, created by `install_deps.sh`. This user is granted necessary permissions (`docker group`, `passwordless sudo`).
- **Dependency Management:** Dependencies are managed via `package.json` and installed using `npm install`. The `.gitignore` file prevents accidental commits of sensitive files like `.env` or `node_modules`.

4. Input Validation and Error Handling:

- NestJS DTOs provide basic structural validation. Explicit validation decorators (`class-validator`) are used in some DTOs (e.g., `UpdateUserDto`).
- Services perform additional validation logic (e.g., checking for empty message content, validating campus existence, checking for duplicate usernames/emails during registration/update).
- The global `HttpExceptionFilter` ensures consistent error responses are sent to the client, preventing potential information leakage through default error pages or stack traces.

5. Identified Risks and Potential Improvements:

- **Passwordless Sudo:** The `ccuser` having passwordless sudo access (`install_deps.sh`) is convenient for automation but increases risk if the account is compromised. This could be restricted further.
- **Keel Authentication:** The Keel deployment currently uses hardcoded basic authentication (`admin:password` in `keel-values.yaml`), which is insecure. This is noted as needing migration to a secrets-based approach.
- **Rate Limiting:** No explicit rate limiting is implemented on API endpoints or WebSocket connections, potentially leaving the application vulnerable to brute-force attacks or denial-of-service.
- **Input Sanitization:** While basic validation exists, more rigorous input sanitization could be added to further prevent injection attacks (e.g., XSS in chat messages or posts).
- **Dependency/Image Scanning:** The CI/CD pipeline does not currently include automated steps for scanning dependencies (NPM packages) or container images for known vulnerabilities. Implementing tools like `npm audit` or integrating Docker Hub scanning/third-party scanners would enhance security.

Overall, a multi-layered approach to security has been implemented, addressing common web application vulnerabilities. However, areas for further hardening exist, particularly around infrastructure permissions, third-party component security, and advanced threat mitigation techniques.

Data Persistence and Modeling

The application utilizes two primary data stores: MongoDB for persistent application data and Redis for session management.

MongoDB

MongoDB serves as the main database, storing the core entities required for the application's functionality. Mongoose is used as the Object Document Mapper (ODM) within the NestJS backend to interact with MongoDB.

- **Role:** Stores persistent information about users, campuses, communication channels, messages, and campus posts.
- **Schemas:** Key data structures are defined using Mongoose schemas:
 - **User (`users.schema.ts`):** Contains user identification (`email`, `username`), authentication (`password` hash), profile details (`fullName`, `major`), a reference to their `Campus` (via `Types.ObjectId`), and arrays storing IDs for social connections (`friends`, `friendRequests`, `sentFriendRequests`, `blockedUsers`). Includes timestamps.

```
export interface IUser {
  _id?: string;
  email: string;
  username: string;
  password: string;
  campus: string | ICampus | Types.ObjectId;
  friendRequests?: string[];
  sentFriendRequests?: string[];
  friends?: string[];
  blockedUsers?: string[];
  createdAt?: Date;
  updatedAt?: Date;
  major?: string;
  fullName?: string;
}
```

- **Campus (`campus.schema.ts`):** Stores campus details (`name`, `city`, `state`, `domain`). Includes a `toJSON` transform to format the output ID.

- **Channel** (`channel.schema.ts`): Defines communication channels with a type ('DM' or 'GROUP'), an optional name, and an array of participant User IDs (participants). Includes timestamps.

```
export interface IChannel {
  _id?: string;
  type: 'DM' | 'GROUP';
  name?: string;
  participants: string[]; // Array of user IDs.
  createdAt?: Date;
  updatedAt?: Date;
}
```

- **Message** (`message.schema.ts`): Represents individual messages within a channel, linking to the channelId and senderId. Stores the content, senderName (denormalized for display), an edited flag, and an array of user IDs who have read the message (readBy). Includes timestamps.

```
export interface IMessage {
  _id?: string;
  channelId: string;
  senderId: string;
  senderName?: string;
  content: string;
  edited?: boolean;
  createdAt?: Date;
  updatedAt?: Date;
  readBy?: string[];
}
```

- **Post** (`post.schema.ts`): Represents campus-wide posts, linked to a campusId and senderId. Stores senderName, senderMajor, content, an array of hashtags, and an array of user IDs who liked the post (likes). Includes timestamps.

```
export interface IPost {
  _id: string;
  campusId: string;
  senderId: string;
  senderName?: string;
  senderMajor?: string;
  content: string;
  hashtags: string[];
  likes: string[]; // Array of user IDs who liked the post.
  createdAt?: Date;
  updatedAt?: Date;
}
```

- **Deployment:** Deployed within Kubernetes using a Deployment (`helm/templates/mongo-deployment.yaml`) configured via the Helm chart. Data persistence is handled by a PersistentVolumeClaim (`helm/templates/mongo-pvc.yaml`). Access within the cluster is managed by a ClusterIP Service (`helm/templates/mongo-service.yaml`).

Redis

Redis is used as a high-performance, in-memory store primarily for session management.

- **Role:** Stores active user session data, enabling users to stay logged in across requests.
- **Implementation:** Integrated via the `connect-redis` library within the `SessionMiddleware`. Session data is keyed by the session ID (derived from the `connect.sid` cookie).
- **Security:** Session data is encrypted using AES-256-CBC (`EncryptionService`) before being written to Redis, as configured in the `SessionMiddleware`'s custom serializer. The Redis instance itself is password-protected, with the password managed via a Kubernetes secret (`REDIS_PASSWORD` in `campus-connect-config-secrets`) and mounted into the Redis deployment (`helm/templates/redis-deployment.yaml`).
- **Deployment:** Deployed as a Kubernetes Deployment (`helm/templates/redis-deployment.yaml`) and accessed via a ClusterIP Service (`helm/templates/redis-service.yaml`). No persistent volume is attached, as session data is ephemeral.

Data Access

Interaction with these databases is abstracted within the backend: * MongoDB operations are handled primarily through Repository classes (e.g., `UsersRepository`, `MessageRepository`) which utilize Mongoose models. * Redis session interactions are managed by the `connect-redis` middleware configured in the `SessionModule`.

This separation ensures that data storage details are encapsulated, and services interact with a consistent data access layer.

Conclusion: Reflections on the Learning Process

This project was a significant undertaking for our group, pushing us well beyond basic application development into the complex world of cloud deployment and infrastructure automation. The learning curve was steep, largely because we hadn't tackled a project of this scale or worked extensively with most of these technologies before.

It was difficult to create a detailed plan from the very beginning simply because we didn't have enough experience with Kubernetes, automated CI/CD, or managing cloud infrastructure to even know what needed detailed planning. We knew backend and frontend basics, but the rest was new ground. This lack of foresight, born from inexperience, meant our development process involved a lot of learning on the fly. As the codebase grew and we integrated more features, things definitely got complicated and messy. Refactoring the backend to use NestJS became essential to manage this complexity and make the code more modular. While it helped significantly, we acknowledge that some parts of the final code still reflect that initial period of building while simultaneously figuring things out.

Kubernetes itself presented major hurdles. With little prior deployment experience, just understanding the core concepts was tough. Then came the practical challenges: configuring things differently for development versus production (especially dealing with Nginx needing 'localhost' sometimes and the CloudLab DNS name other times), debugging workload failures, and getting networking sorted out. Before we secured our domain, we needed to dynamically inject the changing CloudLab hostname (like pcXXX.emulab.net) into our Nginx configuration at deploy time. This forced us to learn Helm templating and Skaffold value injection, adding more layers to learn on top of basic Kubernetes, which often led to frustrating debugging cycles trying to pinpoint issues.

When we couldn't use familiar tools like Docker Watchtower for automatic updates in Kubernetes, we had to research alternatives and implement Keel, which meant learning how to deploy and manage yet another component within the cluster. Securing the application properly also brought new challenges. Setting up our custom domain with HTTPS involved figuring out TLS certificate management and Ingress configuration for the first time.

Secrets management proved particularly difficult on CloudLab. Because experiments clone from a public GitHub repository, we couldn't store secrets in our code. We had to create a GitHub Actions workflow (instantiate.yaml) that used SSH (via `init_node.py`) to inject

secrets (stored securely in GitHub Secrets) into the CloudLab node's environment during startup (startup.sh), so they could then be turned into Kubernetes secrets. This required automating the CloudLab node setup using their Powder/GENI-Lib API, which we found incredibly confusing and lacking clear documentation. Getting this automation working involved a lot of trial-and-error, debugging, and help from language models to interpret the API and adapt examples.

We know that using a major cloud provider like AWS or Azure with tools like Terraform and managed secret services would likely streamline this entire infrastructure setup process considerably. However, working with CloudLab's more bare-bones environment, despite being challenging and sometimes overwhelming, forced us to engage with these problems directly. We had to build the automation for dependency installation, user setup, service startup, secrets injection, and network configuration ourselves. This hands-on struggle gave us a much more fundamental understanding of how these cloud systems work under the hood. We genuinely feel better prepared now to work with any cloud provider because we had to build so much from the ground up here.

On a positive note, using Docker from the start was extremely beneficial for team collaboration, ensuring we all had consistent development environments and avoiding many common dependency issues. We did however, run into a couple merge conflicts, which helped us grow in a team environment. *(and we finally figured out how to exit vim.)*

Overall, while delivering a functional proof-of-concept for Campus Connect, the most significant outcome of this project was the practical learning. We encountered and overcame numerous technical and collaborative hurdles, gaining invaluable experience in cloud-native development, CI/CD, Kubernetes, and managing a complex software project as a team.

Appendix: Resumes

Perez Abhulimen

6108185908 | Boothwyn, PA - 19061 | PABHULIMEN76@GMAIL.COM

WORK EXPERIENCE

PJM Interconnection L.L.C.

Audubon, PA

Cyber Security / Platforms and Client Operations

May 2023 - Aug 2023

- Spearheaded the implementation of advanced security protocols, reducing vulnerabilities and enhancing overall system resilience.
- Assisted the Platforms and Client Operations team in deployment refreshes, flash drive encryption, and Image Processing.
- Assisted a cross-functional team to conduct comprehensive security audits of several Intrusion Detection Systems.
- Followed processes accurately with attention to detail while being responsible for monitoring IP address traffic coming in and out of the network.
- Created simulated phishing emails for Critical Infrastructure Protection (CIP) audits, enhancing organizational resilience against social engineering and deepening knowledge of cybersecurity awareness and compliance.

West Chester University

West Chester, PA

Resident Assistant

Aug 2023 - Present

- Implementing and enforcing all College and Campus Living policies, promoting academic excellence, addressing student needs, and encouraging involvement.
- Promoting the interaction and construction of social networks between the community and residents.
- Create, market, and lead student-centered hall programs to promote social, education, diversity, health, emotional, understanding, and growth.
- Communicating a budget, allocating money for residence hall and floor events, and collaborating with on- and off-campus offices such as counseling, student activities, maintenance, etc.
- Collaborating with orientation staff on first year and transfer student opening week activities.
- Budgeting and allocating money used for residence hall and floor events.
- Developed a positive living environment for residents, often resolving conflicts.

Top Golf

King of Prussia, PA

Bay Host

Oct 2023 - Present

- Facilitated a welcoming atmosphere for over 50 guests daily, enhancing overall guest satisfaction ratings by 50% through attentive service and efficient operations.
- Coordinated with kitchen and bar staff to streamline food and beverage service, resulting in a 10% reduction in wait times and a significant boost in customer retention.
- Trained and mentored new team members on best practices and service standards, leading to an improvement in team performance ratings by 15% within the first 6 months.
- Analyzed guest feedback to identify trends and areas for improvement, successfully addressing concerns that led to an increase in positive online reviews.

EDUCATION

West Chester University

West Chester, PA

Bachelor of Computer Science: Cybersecurity

Graduation Date: Date

LEADERSHIP EXPERIENCE

Dowdy Multicultural Center

West Chester, PA

Peer Mentor

Aug 2023 - Present

- Mentored and provided guidance to at least 6 students in the Educational Opportunity Program, resulting in an increase in program completion rates.
- Developed and implemented tailored academic plans for students, leading to several improvements in GPA averages.
- Collaborated with academic advisors to identify at-risk students and successfully implemented interventions that improved student retention.
- Organized and facilitated weekly study groups for students, resulting in a 20% increase in exam scores.

- Assisted in the planning and execution of EOP events and workshops, contributing to an increase in student participation and engagement.

SKILLS & INTERESTS

VMWare, Computer Security, Windows, Network Security, Java, Big Data Analytics, HTML, CSS, JavaScript,

Skills: CI/CD and Docker, Communication, Crisis Management, Event Planning, Software Engineering, Data Structures and Algorithms, Team Collaboration, Critical Thinking, Leadership, Community Building,

Alex M. Cooper Jr.

U.S. Citizen | cooperjralex@gmail.com | (717) 693-9424 | Github: <https://github.com/AlexCooperJr>

Education

West Chester University of Pennsylvania

B.S. in Computer Science

West Chester, Pennsylvania

Expected Graduation: Spring 2026

- **Related Coursework:** Computer Science III, Calculus I, Discrete Mathematics, Statistics, Data Structures and Algorithms, Computer Security, Computer Systems
- Participated in the 7th and 10th Annual West Chester Programming Contest
- Spring 2024 Dean's List, Fall 2024 Dean's List

University of Delaware

Newark, Delaware

August 2022-May 2023

- **GPA:** 3.45/4.00
- Spring 2023 Dean's List

Projects/STEM Experience

Statistics Tutor

Peer Tutor

West Chester, Pennsylvania

August 2024-Current

- Assisted students in either one on one or small group (less than 4) settings to work through topics from the Statistics I course at the West Chester University Learning Assistance and Resource Center (LARC)
- Covered topics including probability, confidence intervals, tests of hypothesis, and regression
- Reviewed material and developed individualized plans to ensure each student was able learn effectively and communicated between faculty, students, and peers to ensure continuity.
- Participated in training with the College Reading and Learning Association to be certified.

Thermostat Project

Lancaster, Pennsylvania

July 2024

- Programmed an Arduino UNO to take in input from a thermistor and display the temperature onto an Liquid Crystal Display(LCD), change a Tri-color LED to a corresponding color based on range, and start a DC motor depending on the registered temperature.
- Utilized the Arduino IDE to write code for communication between the arduino and components.
- Designed the circuit and implemented the appropriate techniques and components to regulate current outputs for protection of all components.

Stock Price Analyzer

Solo Developer

West Chester, Pennsylvania

December 2023-February 2024

- Built a program using Python that can read comma separated value (CSV) files and uses the info to visualize the information provided and attempts to predict future prices using linear regression.
- Utilized the Pandas library to create DataFrames which store the values to be visualized in graphs and the terminal, these DataFrames are then manipulated to add more detail and to be accessed for different purposes.
- Used the Matplotlib library to create plots and subplots to help visualize the information into a variety of graphs.

- Applied the scikit-learn (sklearn) library to run a linear regression model based on training and testing sets to determine the information's trend.

Weather Website

West Chester, Pennsylvania

Solo Developer

January 2024

- Built a website that can tell the current temperature and weather for any city in the United States.
- Used HTML to build the user interface which has a designated area to enter a city name and used CSS to style the site.
- Utilized my skills and knowledge to integrate the fetching of information using an application programming interface (API) key.
- Used JavaScript to write the functions that locate the coordinates of a given city and then use those coordinates to determine the weather, updating the HTML to display a description, picture, and temperature converted to Fahrenheit.
- Implemented throwing exceptions to prevent errors when a city is misspelled or can not be located.

Encryption/Decryption Program

Newark, Delaware

Solo Developer

February 2023-April 2023

- Built a program using Python which can encrypt messages providing a unique hashkey for each combination of characters, the program can also decrypt messages made using said cypher and confirms the integrity of the messages by verifying the hashkey.
- Integrated my knowledge of the ASCII table for character rotation and manipulation functions.
- Supports user inputs and can be customized to support variations of haskeys in different bases.

Technical Skills

Languages: Python, Java, HTML, JavaScript, CSS, C, C++

Libraries: Pandas, Numpy, Matplotlib, scikit-learn, Java.util

Environments/Tools: Thonny, BlockPy, jGRASP, Visual Studio Code, GitHub, Arduino IDE

Non-Technical Work Experience

Valvoline Instant Oil Change

Lancaster, Pennsylvania

Senior Technician

May 2022-August

2024

- Supervised a team of employees to ensure speed of service, quality work, and safety.
- Interacted with suppliers to ensure supplies and resources were properly managed and available.
- Used a variety of new technologies, and industry standards to perform a number of roles during vehicle servicing.
- Kept track of and issued disciplinary files and ensured accountability for team members.

Resident Assistant Internship and Leadership Exploration(RAILE)

Newark, Delaware

Intern/Mentee

February 2023-May 2023

- Attended a variety of professional workshops that addressed issues commonly seen in work places and resident halls.
- Received on-site training and met with a mentor to build and develop characteristics of a leader.
- Helped coordinate and host events to engage with members of the community.

TYLER GEIGER

Software Developer

✉ tygg513@outlook.com

☎ (267) 500-7205

📍 West Chester, PA

🌐 [linkedin.com/in/tyler-geiger](https://www.linkedin.com/in/tyler-geiger)

WORK EXPERIENCE

Copyright Office – Temporary Employee

ASTM International

📅 2022 - current 📍 Conshohocken, PA

- Successfully registered over 5000 of ASTM's published literary works with the U.S Copyright Office through the ECO system.
- Developed and proposed an automation to the IT Department for the process above using Node.js, the XLSX library, and a Copyright Public Records API.
- Developed numerous ease-of-use automations and shortcuts utilizing Excel Office scripts and Windows PowerShell, significantly improving workflow efficiency, and simplifying tasks for my supervisor.

IT Operations – Temporary Employee

ASTM International

📅 June 2023 - August 2023 📍 Conshohocken, PA

- Application Portfolio Management: Populated an APM (Application Portfolio Management) site by conducting interviews with IT team members to gather and document detailed metadata on applications, including application dependencies, subject matter experts, and infrastructure components (DEV, QA, Prod servers), among other details.

PROJECTS

Software Engineering, Chester County Association for the Blind and Visually Impaired (CCABVI) Website Rework

Team Lead

📅 Sept 2024 - Dec 2024

- Led a website enhancement project, implementing UI/UX improvements focused specifically for accessibility with blind users, including features designed to comply with 508/ADA standards and optimize screen reader navigation.
- Led requirements gathering sessions with a non-technical client, translating their needs into actionable development plans.

SUMMARY

I am a third-year Computer Science student at West Chester University with a passion for software development and problem-solving. I am seeking an internship or research opportunity to apply my technical skills, gain hands-on experience, and contribute to innovative solutions in the field.

SKILLS

- Javascript
- NodeJS, ExpressJS, & MongoDB
- React.js (Familiar, actively learning)
- HTML, CSS
- Git, Github, & Bitbucket
- Excel & MS Office Scripting
- Java
- Problem Solving
- Bash & Powershell
- Visual Studio Code
- SQL

EDUCATION

Diploma

Souderton Area High School

📅 2018 - 2022

📍 Souderton, PA

Bachelor of Sciences

Major in Computer Science, Minor in Applied Statistics

West Chester University

📅 Aug 2022 - current

- Resolved technical issues, including fixing a PayPal integration and identifying hosting configurations for the WordPress-based website.
- Collaborated with teammates, taking a leadership role in front-end development and team communication.

Electronic Copyright Office Registration Automation

Creator

- Automated the registration process on the Copyright Office's Electronic Copyright Office (ECO) system using PuppeteerJS and Node.js.
- Designed and implemented workflows to extract data from Excel spreadsheets, format it for automation, and retrieve prior registration numbers by querying the Copyright Public Records API.
- Designed additional scripts using PuppeteerJS to automate the upload of PDFs from a shared network drive to the ECO system.
- Reduced a previously week-long, sometimes multi-week manual process into an automation that can be configured and executed in a single day, significantly improving efficiency.

📍 West Chester, PA

Awards

- Dean's List, Fall 2024
- Dean's List, Spring 2024

Relevant courses

- Computer Science I, II, & III
- Foundations of CSC
- Data Structures & Algorithms
- Computer Security & Ethics
- Programming Lang Concepts & Paradigms
- Software Engineering
- Database Mgt Systems
- Modern Web Apps Using Server-Side Technologies
- Computer Science Principles
- Data communications & Networking

Kadin Matotek

302-723-2182 | kadinmatcs@gmail.com | linkedin.com/in/kadin-matotek | github.com/kmatotek

EDUCATION

Strath Haven High School

Wallingford, PA

High School Diploma

Graduation: 5/2022

- Clubs and Organizations: Ice Hockey, Track and Field, and Soccer.
- Relevant Coursework: Computer Science Principles

West Chester University of Pennsylvania

West Chester, PA

B.S. in Computer Science

Expected Graduation: 5/2026

- Minor: Mathematics
- GPA: 3.94/4.0
- Clubs and Organizations: Omicron Delta Kappa, Computer Science Club, and Club Ice Hockey.
- Relevant Coursework: Discrete Mathematics, Computer Systems, Computer Security, Data Structures and Algorithms, AI, Software Engineering, Data Communications and Networking, Cloud Computing, Calculus I and II, Linear Algebra and Applied Statistics.

PROJECTS

Email Classifier | *Java*

Oct 2023

- Achieved 89% prediction accuracy in distinguishing legitimate emails from spam across a dataset of 5,000 emails by calculating distances between individual emails and clusters.
- Implemented Java GUI enabling users to assess email authenticity and calculate distances between individual emails and/or email groups.
- Integrated a range of classification techniques, including nearest neighbors and Euclidean distance calculations, to improve adaptability and strengthen analytical precision.

Happy Programming Language | *Java*

Nov 2024

- Designed and developed “Happy”, a custom programming language featuring support for variable assignments, conditional expressions, loops, functions, list operations, and string formatting, aimed at providing a clean and concise syntax compared to other common programming languages.
- Documented the development and syntax through knowledge base documentation, allowing for anyone to learn the syntax, built in functions, and everything needed to know to start developing with this language.

2D Unity Game | *C#, Unity*

Oct 2023

- Developed a 2D video game inspired by *The World’s Hardest Game*, incorporating challenging level design and precise player controls.
- Designed and implemented multiple levels with increasing difficulty to test players’ reflexes and problem-solving skills.
- Conducted play-testing to refine game mechanics and enhance user experience based on feedback.

3D Unity Game | *C#, Unity*

Nov 2023

- Developed a 3D game incorporating custom graphics, custom player scripts, and realistic physics simulations in outer space.
- Developed multiple levels, presenting players progressively challenging obstacles to overcome, ensuring continuous skill progression.
- Conducted user testing to get feedback and implemented improvements through updates.

Facial Recognition | *Python*

Feb 2024

- Engineered a real-time face recognition program achieving over 90% accuracy on a structured dataset, providing efficient visual identification.
- Integrated advanced facial detection and recognition algorithms, utilizing libraries to detect and recognize faces in diverse environments.

EXPERIENCE

D.P. Dough West Chester

Mar - Aug 2023

Delivery Driver

West Chester, PA

- Executed over 1,000 customer deliveries with a strong focus on optimizing delivery routes, resolving logistical challenges, and ensuring timely, accurate order fulfillment through advanced problem-solving techniques.
- Utilized problem-solving skills to streamline order management processes and provide customer service.
- Took orders, served customers, addressed inquiries, resolved any issues to ensure the best customer experience.

Vicky's Place

Jun - Jul 2021

Dish Washer

Swarthmore, PA

- Assisted kitchen staff by ensuring a steady supply of clean cookware and dishware during peak hours.
- Maintained a fast-paced and organized dishwashing station to support smooth kitchen operations.
- Trained new employees on proper dishwashing procedures, safety protocols, and kitchen cleanliness standards to ensure consistency and efficiency.

Fort Delco Gym

Jun - Jul 2020

Gym Custodian

Morton, PA

- Maintained cleanliness and sanitation of workout areas, locker rooms, and equipment to ensure a safe and hygienic environment for both members and staff.
- Conducted routine inspections to identify and address maintenance or cleanliness issues promptly.
- Collaborated with staff to uphold gym standards and provide a positive experience for members.

Open Sky Energy

Aug - Nov 2019

Social Media Manager

Swarthmore, PA

- Collaborated with the team to develop marketing strategies and improve social media outreach.
- Developed and scheduled posts to promote solar energy solutions, company updates, and customer success stories.
- Managed and created content for the company's social media platforms, increasing engagement and brand visibility.

VOLUNTEERING

Media Food Bank

Mar - Aug 2023

Volunteer

Media, PA

- Assisted customers by taking food orders and providing a welcoming, supportive environment.
- Retrieved and organized food items based on customer orders to ensure accuracy and efficiency.
- Worked collaboratively with volunteers and staff to maintain smooth food distribution operations.

TECHNICAL SKILLS

Proficient Languages: Java, Python

Familiar Languages: Haskell, C, R, C#, SQL, HTML, CSS, Swift, LaTeX

Developer Tools: R Studio, SPSS, Git, VS Code, Linux, Zshell, Windows, Mac OS, Stack Overflow

Other Familiar Tools: Microsoft Office Suite, Unity